

MODULE 22

LSTM Next Word Prediction Project

Lectures Covered

- Lecture 1 — Overview of Project Task
- Lecture 2 — Getting Started with Project
- Lecture 3 — Tokenization and Text Prep for Modelling
- Lecture 4 — Creating Model for Next Word Prediction
- Lecture 5 — Streamlit UI for App
- Lecture 6 — Predicting Through Model and Pushing Code to GitHub

Lecture 1: Overview of Project Task

This module builds a production-ready next-word prediction app powered by an LSTM network — the same core mechanism behind early large language models. The project demonstrates the full ML pipeline from raw text to a deployed web application.

What We Are Building

Project at a Glance

Task: Next-word prediction — given N words, predict the most probable next word.
Model: LSTM with Embedding layer (deeper and more capable than the Module 20 SimpleRNN).
Dataset: Shakespeare's complete works — rich vocabulary, complex long-range dependencies.
UI: Streamlit app with real-time word prediction and top-k suggestions.
Deployment: GitHub (Git LFS for model files) + Streamlit Cloud.

Why LSTM over Simple RNN?

Capability	Simple RNN LSTM
Long-range context	Degrades beyond ~20 tokens Retains context over 100+ tokens
Vanishing gradient	Severe — limits learning Mitigated by cell state highway
Gate mechanism	None 3 gates: Forget, Input, Output
Next-word quality	Locally coherent only Globally coherent over long phrases
Model complexity	Low Higher — requires more data & compute

Why This Is an LLM Foundation

- LSTM-based language models were the state-of-the-art before Transformers (pre-2017).
- GPT-1 replaced LSTM with Transformer self-attention — but the task is identical: predict next token.
- Understanding LSTM next-word prediction gives direct intuition for how GPT and modern LLMs work.
- The training data pipeline, tokenisation, and inference logic are essentially the same.

Project File Structure

```
lstm-next-word/  
├── data/  
│   └── shakespeare.txt          # ~5MB raw corpus  
├── models/  
│   └── lstm_next_word.h5        # trained model (Git LFS)  
├── tokenizer.pkl                # saved Keras tokenizer  
├── app.py                       # Streamlit application  
├── train.py                     # training script  
├── requirements.txt  
└── .gitattributes               # Git LFS config
```

Lecture 2: Getting Started with Project

Environment Setup

```
# Create virtual environment  
python -m venv lstm_env  
source lstm_env/bin/activate      # Mac/Linux  
lstm_env\Scripts\activate        # Windows
```

```
# Install dependencies
pip install tensorflow numpy scikit-learn streamlit pickle5

# requirements.txt
tensorflow==2.15.0
numpy==1.24.3
scikit-learn==1.3.0
streamlit==1.28.0
```

Loading and Exploring the Corpus

```
with open('data/shakespeare.txt', 'r', encoding='utf-8') as f:
    raw_text = f.read()

# Basic statistics
words = raw_text.lower().split()
unique_words = set(words)

print(f'Total characters : {len(raw_text):,}')
print(f'Total words      : {len(words):,}')
print(f'Unique words     : {len(unique_words):,}')
print(f'Sample           : {raw_text[:200]}')
```

Why Shakespeare as a Corpus?

Property	Value for Training
Vocabulary richness	~25,000+ unique words — broad lexical coverage
Sentence variety	Questions, commands, monologues — diverse syntax patterns
Long-range deps	Characters, themes span entire plays — ideal for LSTM testing
Size	~5 MB — small enough to train locally in reasonable time
Public domain	No licensing issues — freely usable

Training time estimate

Shakespeare corpus + LSTM(150 units) + Embedding(100) on CPU: ~30-60 min / epoch.

On GPU (Colab T4): ~3-5 min / epoch.

Recommended: use Google Colab with GPU runtime for faster iteration.

Save model after each epoch with ModelCheckpoint to avoid losing progress.

Lecture 3: Tokenization and Text Prep for Modelling

Text must be converted into structured numerical sequences before the LSTM can process it. This lecture covers the complete preprocessing pipeline — more advanced than Module 20, with longer sequences to leverage LSTM's long-range memory.

Step 1 — Clean and Normalise

```
import re

def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-z\s|', '', text) # remove punctuation & digits
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

```
text = clean_text(raw_text)
words = text.split()
print(f'Cleaned word count: {len(words):,}')
```

Step 2 — Fit Tokenizer

```
from tensorflow.keras.preprocessing.text import Tokenizer
import pickle

tokenizer = Tokenizer()
tokenizer.fit_on_texts([text])

vocab_size = len(tokenizer.word_index) + 1 # +1 for padding token (index 0)
print(f'Vocabulary size: {vocab_size:,}')
```

Convert full text to integer sequence

```
token_list = tokenizer.texts_to_sequences([text])[0]
print(f'Token sequence length: {len(token_list):,}')
```

Save tokenizer for later inference

```
with open('tokenizer.pkl', 'wb') as f:
    pickle.dump(tokenizer, f)
```

Step 3 — Create N-gram Sequences

Use a sliding window of SEQ_LEN=20 (longer than Module 20's 10) to exploit LSTM's ability to learn longer dependencies:

```
import numpy as np

SEQ_LEN = 20 # 20 input words -> predict the 21st
sequences = []

for i in range(SEQ_LEN, len(token_list)):
    seq = token_list[i - SEQ_LEN : i + 1] # 21 tokens
    sequences.append(seq)

sequences = np.array(sequences) # shape: (N, 21)
X = sequences[:, :-1] # shape: (N, 20) -- inputs
y = sequences[:, -1] # shape: (N,) -- targets

print(f'Training sequences: {len(sequences):,}')
```

```
print(f'X shape: {X.shape} | y shape: {y.shape}')
```

Step 4 — One-Hot Encode Labels and Split

```
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split

y_cat = to_categorical(y, num_classes=vocab_size) # shape: (N, vocab_size)

X_train, X_val, y_train, y_val = train_test_split(
    X, y_cat, test_size=0.2, random_state=42
)

print(f'X_train: {X_train.shape} | y_train: {y_train.shape}')
```

```
print(f'X_val : {X_val.shape} | y_val : {y_val.shape}')
```

Pipeline summary

raw text -> clean_text() -> Tokenizer.fit() -> integer token list
 token list -> sliding window (size 20) -> (X, y) pairs

y -> to_categorical() -> one-hot labels -> 80/20 train/val split
 SEQ_LEN=20 chosen to test LSTM's long-range memory (vs 10 in SimpleRNN module).

Lecture 4: Creating Model for Next Word Prediction

The LSTM model uses an Embedding layer for dense word representations, followed by two stacked LSTM layers to capture both short and long-range patterns, and a Dense softmax head for next-word prediction.

Model Architecture

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout

VOCAB_SIZE = vocab_size
SEQ_LEN = 20
EMBEDDING_DIM = 100
LSTM_UNITS_1 = 150
LSTM_UNITS_2 = 100

model = Sequential([
    # Layer 1: Embedding
    # (batch, 20) -> (batch, 20, 100)
    Embedding(VOCAB_SIZE, EMBEDDING_DIM, input_length=SEQ_LEN),

    # Layer 2: First LSTM - return sequences for stacking
    LSTM(LSTM_UNITS_1, return_sequences=True),

    # Layer 3: Dropout for regularisation
    Dropout(0.2),

    # Layer 4: Second LSTM - extract higher-level patterns
    LSTM(LSTM_UNITS_2, return_sequences=False),

    # Layer 5: Dropout
    Dropout(0.2),

    # Layer 6: Dense output
    Dense(VOCAB_SIZE, activation='softmax') # probability over all words
])

model.summary()
```

Layer-by-Layer Explanation

Layer	Output Shape	Purpose
Embedding(V, 100)	(batch, 20, 100)	Map word indices to dense 100-dim vectors
LSTM(150, seq=True)	(batch, 20, 150)	Capture sequential patterns, pass all steps to next LSTM
Dropout(0.2)	(batch, 20, 150)	Randomly zero 20% of units to prevent overfitting
LSTM(100, seq=False)	(batch, 100)	Higher-level abstraction; output only final hidden state
Dropout(0.2)	(batch, 100)	Second regularisation layer
Dense(V, softmax)	(batch, V)	Probability distribution over entire vocabulary

Why Two LSTM Layers?

- First LSTM processes raw word embeddings -> learns basic syntactic patterns (verb follows noun, etc.).

- Second LSTM processes the first LSTM's output -> learns semantic patterns and thematic context.
- Stacking increases model depth without proportionally increasing parameters.

Compiling and Training

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau

model.compile(
    optimizer = 'adam',
    loss      = 'categorical_crossentropy',
    metrics   = ['accuracy']
)

callbacks = [
    EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True),
    ModelCheckpoint('models/lstm_next_word.h5', monitor='val_loss',
                    save_best_only=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3,
                      min_lr=1e-6, verbose=1),
]

history = model.fit(
    X_train, y_train,
    epochs      = 100,
    batch_size  = 256,
    validation_data = (X_val, y_val),
    callbacks   = callbacks
)

print(f'Best val accuracy: {max(history.history["val_accuracy"]):.4f}')
```

Training tips

Use batch_size=256 or 512 — larger batches speed up training on GPU.

ReduceLROnPlateau cuts the LR when val_loss plateaus -> fine-grained convergence.

EarlyStopping with restore_best_weights=True ensures you keep the best checkpoint.

On Colab: Runtime -> Change runtime type -> GPU (T4) for ~10x speedup.

Lecture 5: Streamlit UI for App

The Streamlit app provides an interactive interface where users enter a seed phrase and receive real-time next-word predictions with probability scores. It also supports multi-word generation by feeding predictions back as input.

Complete app.py

```
import streamlit as st
import numpy as np
import pickle
from tensorflow.keras.models import load_model

# — Page config —————
st.set_page_config(page_title='LSTM Next Word Predictor',
                    page_icon='brain', layout='centered')

# — Load assets (cached) —————
@st.cache_resource
def load_assets():
    tok = pickle.load(open('tokenizer.pkl', 'rb'))
    model = load_model('models/lstm_next_word.h5')
    return tok, model
```

```
tokenizer, model = load_assets()
SEQ_LEN = 20

# --- Core prediction function ---
def predict_next(seed_text, top_k=5):
    tokens = tokenizer.texts_to_sequences([seed_text.lower()])[0]
    tokens = tokens[-SEQ_LEN:] # keep last SEQ_LEN
    tokens = [0] * (SEQ_LEN - len(tokens)) + tokens # left-pad with zeros
    probs = model.predict(np.array([tokens]), verbose=0)[0]
    top = probs.argsort()[-top_k:][::-1]
    return [(tokenizer.index_word.get(i, '?'), float(probs[i])) for i in top]

# --- Generate N words autoregressively ---
def generate_text(seed, n_words):
    result = seed
    for _ in range(n_words):
        word, _ = predict_next(result, top_k=1)[0]
        result = result + ' ' + word
    return result

# --- UI ---
st.title('LSTM Next Word Predictor')
st.markdown('*Trained on Shakespeare - powered by LSTM*')
st.markdown('---')

seed = st.text_input('Enter seed phrase:',
                    value='to be or not to be that is the question')
top_k = st.slider('Top-K predictions', 1, 10, 5)
n_gen = st.slider('Words to generate', 1, 30, 10)

col1, col2 = st.columns(2)

with col1:
    if st.button('Predict Next Word', type='primary'):
        st.subheader('Top Predictions')
        for word, prob in predict_next(seed, top_k):
            st.progress(prob, text=f'{word} {prob:.2%}')

with col2:
    if st.button('Generate Text'):
        st.subheader('Generated Continuation')
        output = generate_text(seed, n_gen)
        st.info(output)

st.markdown('---')
st.caption(f'Model: Stacked LSTM | Seq len: {SEQ_LEN} | Corpus: Shakespeare')
```

Key Streamlit Patterns

Pattern	Code Purpose
@st.cache_resource	Wraps load_assets() -> model loads once, stays in memory across users
st.columns(2)	Side-by-side layout for predict vs generate buttons
st.progress(prob)	Visual probability bar for each predicted word
st.slider()	Interactive top_k and n_words controls
st.info(text)	Styled text box for displaying generated output

Lecture 6: Predicting Through Model and Deployment

Inference Deep Dive

Understanding exactly how the model converts a seed phrase into a probability distribution — and how top-k sampling and greedy decoding differ.

```

import numpy as np

def predict_next_word(model, tokenizer, seed_text, top_k=5, seq_len=20):
    # 1. Tokenise seed
    tokens = tokenizer.texts_to_sequences([seed_text.lower()])[0]

    # 2. Truncate to last seq_len tokens (if seed is longer)
    tokens = tokens[-seq_len:]

    # 3. Left-pad with zeros to reach seq_len
    tokens = [0] * (seq_len - len(tokens)) + tokens

    # 4. Model forward pass
    arr = np.array([tokens]) # shape: (1, seq_len)
    probs = model.predict(arr, verbose=0)[0] # shape: (vocab_size,)

    # 5. Rank and return top-k
    top_indices = probs.argsort()[-top_k:][::-1]
    return [(tokenizer.index_word.get(i, '<UNK>'), float(probs[i]))
            for i in top_indices]

# Test
seed = 'to be or not to be that is the question wherefore'
preds = predict_next_word(model, tokenizer, seed)
for word, prob in preds:
    print(f'{word:<20} {prob:.4f}')

```

Decoding Strategies

Strategy	How It Works Output Quality
Greedy (top-1)	Always pick the highest probability word Deterministic; can be repetitive
Top-K sampling	Sample randomly from top K words Varied; more creative output
Temperature	Divide logits by T before softmax; $T < 1$ =sharp, $T > 1$ =flat Controls creativity
Beam search	Track top-B sequences at each step Best for translation/summarisation

```

# Temperature sampling
def sample_with_temperature(probs, temperature=1.0):
    probs = np.array(probs, dtype='float64')
    probs = np.log(probs + 1e-8) / temperature # apply temperature
    probs = np.exp(probs)
    probs = probs / probs.sum() # renormalise
    return np.random.choice(len(probs), p=probs)

# T=0.5 -> conservative, high-probability words
# T=1.0 -> sample from raw distribution (default)
# T=1.5 -> creative but less coherent

```

GitHub Deployment — Git LFS for Model Files

```

# Install Git LFS
git lfs install
git lfs track '*.h5'
git lfs track '*.pkl'

# Commit and push
git add .gitattributes
git add models/ tokenizer.pkl app.py requirements.txt train.py
git commit -m 'LSTM next-word predictor with Streamlit app'
git push origin main

# Deploy on Streamlit Cloud
# 1. Go to share.streamlit.io
# 2. Connect GitHub -> select repo, branch, app.py

```



```
# 3. Click Deploy
# 4. Live at: https://your-username-appname.streamlit.app
```

Deployment Checklist

Item	Requirement
requirements.txt	tensorflow, numpy, scikit-learn, streamlit — pinned versions
.gitattributes	*.h5 and *.pkl tracked by Git LFS before adding model files
Model path	Relative path 'models/lstm_next_word.h5' — not absolute
@st.cache_resource	On both load_model() and tokenizer load — prevents re-loading
SEQ_LEN constant	Must match training value (20) — mismatch causes wrong predictions
tokenizer.pkl	Committed to repo — essential for mapping words to indices

Alternative: Download model at runtime

If .h5 file is too large even for Git LFS quota, host on Google Drive.
Use gdown in @st.cache_resource to download on first cold start:
gdown.download('https://drive.google.com/uc?id=FILE_ID', 'models/lstm.h5')
This keeps the GitHub repo small while still serving the model.

End of Module 22 Notes — Generative AI Course